



HiBT: From Hierarchical Concepts to Behavior Trees for Reliable Robot Task Planning

Anonymous ACL submission | Project Website: <https://anonymous-2026.github.io/HiBT>

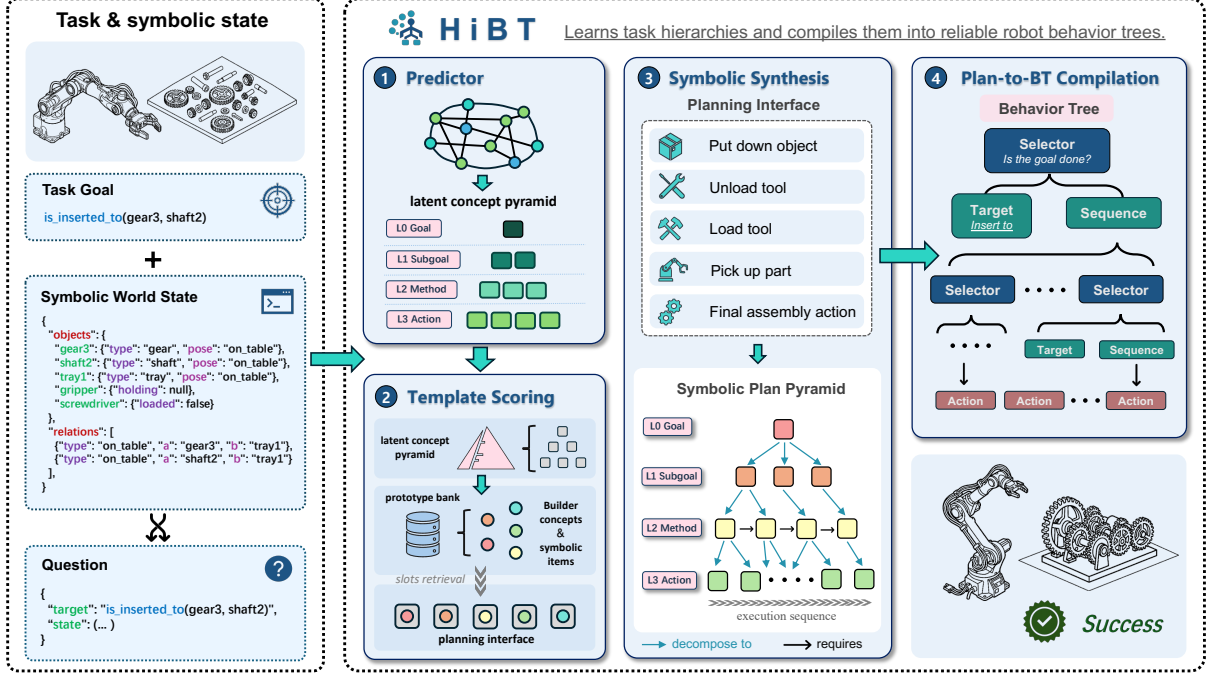


Figure 1: Overview. The planner predicts a hierarchical concept interface from the task instruction and initial state, completes dependency-aware support conditions, and compiles the repaired plan into an executable Behavior Tree.

Abstract

Long-horizon robotic assembly requires planners to reason over object relations, tool use, and ordered preconditions while producing executable control structures. However, most LLM-based planners rely on flat token-by-token generation, which does not explicitly capture the dependencies required for Behavior Tree (BT) synthesis. We propose HiBT, which formulates BT synthesis as hierarchical concept planning. HiBT predicts hierarchical concepts from the task goal and symbolic world state, induces a planning interface through prototype-guided matching, applies deterministic realization and dependency repair, and compiles the resulting symbolic plan into a BT. On a 60-instance language-conditioned assembly benchmark using four open-source LLM backbones, HiBT achieves 60/60 executability, logical coherence, and task success for each backbone, while reducing generation time to 0.35–0.45 seconds. In LIBERO rollouts, HiBT reaches 89.2% final-goal success on LIBERO-Long with shorter rollouts, and 72.3% strict success on the dependency-intensive LIBERO-Hier suite, outperforming

flat-instruction and action-sequence guidance. These results show that a dependency-aware hierarchical interface supports both executable BT synthesis and long-horizon embodied execution.

1 Introduction

Language-conditioned robotic assembly requires a planner to translate task instructions and symbolic world states into executable control structures while preserving object relations, tool states, and ordered preconditions. This is challenging in long-horizon tasks involving tool switching, object release and re-grasp, and intermediate dependencies. For instance, an insertion action may require the correct tool to be attached and the target part to be grasped before execution.

Behavior Trees (BTs) provide an executable representation with explicit condition checks and reusable control substructures (Iovino et al., 2020, 2023). Prior work has studied BT construction through evolutionary computation, reinforcement learning, and structured synthesis (Colledanchise et al., 2019; Banerjee, 2018; Li et al., 2021).

Recent LLM-based planners have generated action sequences, programs, or BTs from language-conditioned inputs (Singh et al., 2023; Song et al., 2023; Driess et al., 2023; Zhou et al., 2024; Ao et al., 2025; Merino-Fidalgo et al., 2025; Ichter et al., 2023; Yao et al., 2023b; Wang et al., 2023a). However, direct autoregressive generation does not explicitly enforce the dependency relations required for executable BT synthesis (Wang et al., 2023b). An action-sequence intermediate can improve ordering, but it still does not specify which stable subgoals support each grounded action.

We propose HiBT, which formulates language-conditioned BT synthesis as hierarchical concept planning. Rather than directly generating a final tree, HiBT predicts a hierarchical latent representation, induces a structured planning interface through prototype-guided matching, repairs missing action dependencies under deterministic symbolic rules, and compiles the repaired plan into an executable BT. The learned interface guides task-compatible structure selection, while symbolic realization enforces the tool, grasp, and ordering conditions required for execution. This design is informed by recent work on hierarchical and planning-oriented reasoning in LLMs (Wei et al., 2022; Yao et al., 2023a; Zhou et al., 2023; Khot et al., 2023; Hao et al., 2023; Chen et al., 2023a; Lyu et al., 2023).

Our contributions are threefold. First, we formulate BT synthesis as hierarchical concept planning followed by symbolic realization, making subgoal dependencies explicit before compilation rather than leaving them implicit in direct generation. Second, we introduce HiBT, which combines hierarchical latent prediction, prototype-guided interface induction, dependency-closing repair, and deterministic BT compilation to construct dependency-consistent executable plans. Third, we evaluate HiBT on a 60-instance symbolic robotic-assembly benchmark across four open-source LLM backbones, together with component ablations and interface perturbations. In the evaluated setting, HiBT achieves full executability, logical coherence, and task success for all tested backbones while reducing generation latency relative to direct-BT and action-sequence baselines; the ablations further indicate that both the learned interface and symbolic realization contribute to these improvements.

2 Related Work

Behavior Trees (BTs) are widely used as executable control representations for robot task planning (Iovino et al., 2020). Prior synthesis work has learned BTs for autonomous agents (Colledanchise et al., 2019), acquired BTs from robot control policies (Banerjee, 2018), and combined reactive task-and-motion planning with formal specifications (Li et al., 2021). These studies establish BTs as practical execution interfaces, but they mainly construct trees from policies, optimization procedures, or formal models rather than from language-conditioned assembly instructions with dense action dependencies.

LLM-based robot planners map language to action sequences, symbolic plans, or program-like outputs (Singh et al., 2023; Song et al., 2023; Rana et al., 2023; Hazra et al., 2024; Chen et al., 2023b; Yao et al., 2023b). Several works directly target BT generation from language, including hybrid BT planning from human instructions (Chen et al., 2024), one-step or adaptive LLM-BT generation (Lykov and Tsetserukou, 2023; Zhou et al., 2024; Izzo et al., 2024), and execution-aware BT synthesis (Ao et al., 2025; Merino-Fidalgo et al., 2025). These approaches show that language models can produce BT-oriented plans, but many still decode the final tree, or a closely related textual plan, as the primary output. As a result, subgoal dependencies and execution constraints are not always represented as explicit intermediate structure before compilation.

HiBT is also related to work on structured reasoning and planning interfaces. Decompositional prompting and planning-oriented LLM methods (Zhou et al., 2023; Khot et al., 2023; Hao et al., 2023; Liu et al., 2023; Liang et al., 2023; Wang et al., 2023a; Gao et al., 2023; Chen et al., 2023a; Lyu et al., 2023) motivate solving complex tasks through intermediate states or executable programs, while recent hierarchical or latent-reasoning methods introduce layered or hidden representations (Gui et al., 2025; Huang et al., 2026; Su et al., 2025; Shen et al., 2025; Amos et al., 2026). HiBT adapts this idea to executable BT synthesis: the model predicts a hierarchical planning interface, and deterministic symbolic realization turns that interface into a dependency-consistent BT for robotic assembly.

3 Method

Given a language-conditioned assembly instance, HiBT produces an executable Behavior Tree through four stages. First, an inference-time Predictor maps the task instruction and symbolic initial state to a fixed-geometry hierarchical latent pyramid. Second, prototype-guided decoding induces a planning interface consisting of a retrieved structural template and slot-level symbolic candidates. Third, a deterministic realization operator constructs an initial grounded plan and applies dependency closure to satisfy missing tool, grasp, and assembly preconditions. Finally, a recursive compiler transforms the dependency-aware method nodes in the repaired plan pyramid into an executable BT. BT compilation operates on grounded method nodes rather than on the auxiliary linearized action sequence.

Figure 2 illustrates this process on a representative gear-insertion instance: HiBT expands the input state into stable support conditions, realizes the corresponding methods as grounded actions, and returns a dependency-aware action trace that first clears and switches the occupied tool before executing the final insertion. Table 1 summarizes the main objects used in the method.

Symbol / Module	Definition
$\mathcal{I}$	Language-conditioned assembly instance.
u, s_0, γ	Task instruction, initial symbolic state, and target predicate.
$\mathcal{D}$	Symbolic assembly domain with predicates, actions, preconditions, and effects.
$\mathbf{n}$	Slot-count vector over the concept levels.
N	Total number of latent concept slots.
h_ϕ	Training-time Builder that extracts target concept pyramids from annotated plans.
f_θ	Inference-time Predictor that maps (u, s_0) to a predicted concept pyramid.
Ξ	Hierarchical planning interface induced by prototype-guided decoding.
P	Dependency-aware plan pyramid with goal, stable subgoals, method nodes, and action sequence.
$\mathcal{T}$	Output Behavior Tree.

Table 1: Main symbols used in the method.

3.1 Problem Formulation

We represent a language-conditioned assembly task as

$$\mathcal{I} = (u, s_0, \gamma, \mathcal{D}), \quad (1)$$

where u is the natural-language instruction, $s_0 \subseteq \mathcal{Q}$ is the initial symbolic state, $\gamma \in \mathcal{Q}$ is the target predicate, and $\mathcal{D}$ is the symbolic assembly domain. We write $\mathcal{D} = (\mathcal{Q}, \mathcal{A}, \mathcal{P}, \mathcal{E}^+, \mathcal{E}^-)$, where $\mathcal{Q}$ is the predicate universe and $\mathcal{A}$ is the set of grounded actions. For each grounded action $a \in \mathcal{A}$, the precon-

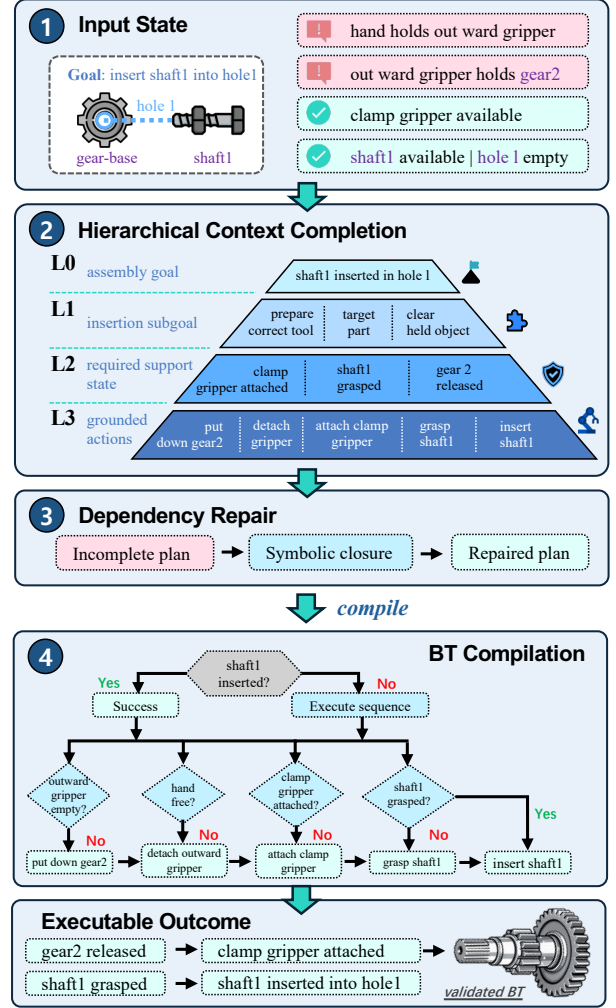


Figure 2: Running example of HiBT on a gear-insertion task. HiBT completes the hierarchical context, repairs missing support conditions, and emits a dependency-aware action trace before final BT compilation.

ditions and positive/negative effects are $P_a \subseteq \mathcal{Q}$, $E_a^+ \subseteq \mathcal{Q}$, and $E_a^- \subseteq \mathcal{Q}$. When $P_a \subseteq s$, the symbolic transition is

$$\delta(s, a) = (s \setminus E_a^-) \cup E_a^+. \quad (2)$$

The method outputs a BT $\mathcal{T}$ that is executable under the shared symbolic runtime and reaches γ under symbolic execution from s_0 . We evaluate these requirements empirically through Exec, LC, and SR rather than treating task success as a purely formal guarantee. Appendix B expands this domain definition, including the symbolic transition and leaf-evaluation semantics in Eqs. 17 and 31.

3.2 Hierarchical Plan Pyramid

HiBT uses the four-level symbolic plan pyramid in Eq. 3:

$$P = (\gamma, \mathcal{S}, \mathcal{M}, \pi), \quad (3)$$

where γ is the final target predicate, $\mathcal{S} = \{q_j\}_{j=1}^J$ is the set of stable subgoals, $\mathcal{M} = \{m_k\}_{k=1}^K$ is the set of grounded method nodes, and $\pi = \langle a_k \rangle_{k=1}^K$ is a linearized action sequence retained for reporting and action-leaf consistency evaluation. Each method node is defined as

$$m_k = (q_k, a_k, U_k). \quad (4)$$

Here $q_k \in \{\gamma\} \cup \mathcal{S}$ is the condition achieved by $a_k \in \mathcal{A}$, and $U_k \subseteq \mathcal{S}$ contains the stable subgoals that should hold before executing a_k . Compilation operates on the dependency structure encoded by $\mathcal{M}$, whereas π is used only for reporting and evaluation.

3.3 Concept Interface

The learned component predicts a fixed-geometry latent concept pyramid (Wang et al., 2023a; Lyu et al., 2023). Let $\mathbf{n}$ denote the slot-count vector and N the total number of latent slots:

$$\begin{aligned} \mathbf{n} &= (n_0, n_1, n_2, n_3), \\ N &= \sum_{\ell=0}^3 n_\ell. \end{aligned} \quad (5)$$

In this work, $\mathbf{n} = (1, 4, 5, 5)$ and $N = 15$. At level ℓ , the concept tensor is

$$\mathbf{Z}^{(\ell)} = [\mathbf{z}_i^{(\ell)}]_{i=1}^{n_\ell} \in \mathbb{R}^{n_\ell \times d}, \quad (6)$$

and the full latent pyramid is $\mathbf{Z} = \{\mathbf{Z}^{(0)}, \mathbf{Z}^{(1)}, \mathbf{Z}^{(2)}, \mathbf{Z}^{(3)}\}$. The training-time Builder and inference-time Predictor are written as

$$\mathbf{C}^* = h_\phi(u, s_0, P^*), \quad \hat{\mathbf{Z}} = f_\theta(u, s_0), \quad (7)$$

where P^* is an annotated symbolic plan pyramid, $\mathbf{C}^*$ is the target concept pyramid, and $\hat{\mathbf{Z}}$ is the predicted concept pyramid. The Predictor is trained with

$$\mathcal{L} = \lambda_Z \mathcal{L}_Z + \lambda_\pi \mathcal{L}_\pi, \quad (8)$$

where $\mathcal{L}_Z$ is the per-level concept reconstruction loss and $\mathcal{L}_\pi$ is the teacher-forced loss on the annotated symbolic-plan serialization. In this instantiation, the concept loss is the per-level mean squared error between predicted and Builder concept slots; the sequence loss is used during training, while inference uses the predicted concept pyramid for planning. We use concept to denote a level- and slot-aligned latent representation supervised by annotated plan pyramids; individual latent slots are

not assumed to correspond to human-interpretable concept labels.

Prototype-guided decoding maps the predicted concept pyramid to a structured planning interface (Gao et al., 2023; Chen et al., 2023a). The prototype bank is

$$\mathcal{B} = \{b_j = (\mathbf{C}_j, \tau_j, \Sigma_j)\}_{j=1}^M, \quad (9)$$

where $\mathbf{C}_j$ is a Builder concept pyramid, τ_j is its structural template, and Σ_j is the associated symbolic plan schema. Template matching uses the template score in Eq. 10:

$$s_\tau(b_j) = \frac{1}{4} \sum_{\ell=0}^3 \cos(\mathbf{v}(\hat{\mathbf{Z}}^{(\ell)}), \mathbf{v}(\mathbf{C}_j^{(\ell)})), \quad (10)$$

where $\mathbf{v}(\cdot)$ vectorizes a level tensor. The selected bank entry is indexed by $j^* = \arg \max_j s_\tau(b_j)$ and $\tau^* = \tau_{j^*}$. For each level and slot, the decoder retrieves the nearest symbolic item under the selected template:

$$r_i^{(\ell)} = \arg \max_{p \in \mathcal{B}_i^{(\ell)}(\tau^*)} \cos(\hat{\mathbf{z}}_i^{(\ell)}, \mathbf{c}(p)). \quad (11)$$

Let Φ denote prototype-guided decoding. The induced interface is

$$\begin{aligned} \Xi &= \Phi(\hat{\mathbf{Z}}; \mathcal{B}) = (\tau^*, \mathcal{R}), \\ \mathcal{R} &= \{r_i^{(\ell)}\}_{\ell, i}. \end{aligned} \quad (12)$$

This decoding stage corresponds to Algorithm 1; Appendix B gives the prototype-bank construction and slot-retrieval details.

3.4 Dependency Closure

The deterministic component realizes the interface under the symbolic domain and closes missing dependencies before compilation (Lyu et al., 2023). We use $\rho_{\mathcal{D}}$ for initial realization, $\kappa_{\mathcal{D}}$ for closure, and Λ for lifting:

$$\begin{aligned} \pi_0 &= \rho_{\mathcal{D}}(\gamma, s_0, \Xi), \\ \pi^* &= \kappa_{\mathcal{D}}(\pi_0, s_0), \\ \tilde{P} &= \Lambda(\gamma, \pi^*). \end{aligned} \quad (13)$$

The initial sequence π_0 is instantiated from the retrieved template and slot-level retrieval signals. Dependency closure then checks each action against the current symbolic state. For an action a in state s , the missing dependency set is

$$\Delta(a, s) = P_a \setminus s. \quad (14)$$

Whenever $\Delta(a, s) \neq \emptyset$, the closure operator inserts domain-supported actions that establish the missing conditions before a is executed. The closure operator uses deterministic domain-rule dispatch rather than open-ended symbolic search. In the assembly domain, when a candidate assembly action requires an incompatible tool state, dependency closure inserts release, detach, attach, and grasp actions before the final assembly action. The repaired sequence is then lifted into the plan pyramid $\tilde{P} = (\gamma, \mathcal{S}, \mathcal{M}, \pi)$, where stable subgoals record support conditions that are explicitly checked or re-established by the compiled BT under repeated ticking. Algorithm 2 and Eqs. 28–29 give the full closure recurrence in the appendix.

The same interface can be applied to embodied manipulation benchmarks by replacing the assembly-domain request parser with a benchmark adapter. For LIBERO-style tasks, the adapter reads the task language, object inventory, initial predicates, and goal predicates from the benchmark specification, then normalizes them into symbolic manipulation primitives such as placing an object in or on a target receptacle, opening or closing an articulated object, or changing the state of an appliance. HiBT still produces a high-level dependency-aware subgoal structure, while a vision-language-action carrier executes the low-level control policy for each subgoal. This extension changes the execution substrate, not the role of the planning interface: symbolic planning success and physical rollout success are reported separately.

3.5 BT Compilation

The compiler recursively maps the repaired plan pyramid to a BT:

$$\mathcal{T} = C(\tilde{P}). \quad (15)$$

For any condition $q \in \{\gamma\} \cup \mathcal{S}$, let $m(q)$ denote the method node that achieves q , and let $U_{m(q)}$ be its required stable subgoals. The branch B_q achieves q after these required stable subgoals have been compiled. The recursive compilation rule is

$$\begin{aligned} B_q &= \sigma_{\wedge} \left[[C(r)]_{r \in U_{m(q)}}, \psi(a_{m(q)}), \alpha(a_{m(q)}) \right], \\ C(q) &= \sigma_{\vee} [\chi(q), B_q], \end{aligned} \quad (16)$$

and the final tree is $\mathcal{T} = C(\gamma)$. Thus every goal or stable subgoal is represented by a selector with a target-check branch and an achievement branch.

Every method node first compiles its required stable subgoals, then checks the action preconditions, and finally emits the grounded action leaf.

Proposition 1. Under an acyclic repaired dependency graph and valid grounded action schemas, the compiler deterministically produces a schema-valid BT whose action leaves are induced exclusively from the grounded actions stored in $\mathcal{M}$. This property concerns structural validity and dependency preservation; task success remains empirically evaluated through symbolic execution. Appendix B gives the full formal specification, proof sketches, leaf semantics, and Algorithms 1–3.

4 Experiments

4.1 Task Setting and Benchmark

We evaluate BT synthesis as a language-conditioned robot task planning problem. Each test instance provides a task goal together with an initial symbolic world state, and the system must output an executable BT that can be validated by symbolic simulation. The main benchmark used in this paper is a 60-instance symbolic robotic-assembly benchmark. It covers four domains: Gearset (1 instance), Gearset-1 (44), Chair (9), and Lamp (6). It also spans four difficulty buckets: easy (22), medium (13), hard (11), and very hard (14). Appendix A describes the task-family distribution, difficulty partition, and exclusion protocol.

We additionally conduct a targeted interface-robustness study on the same 60-instance set using Qwen2.5-7B-Instruct.

The same protocol design also supports a LIBERO manipulation extension, where benchmark task specifications are converted into symbolic planning requests and the resulting subgoals can guide a low-level VLA carrier. Because this setting adds physical rollout outcomes on top of symbolic checks, the completed assembly benchmark remains the main symbolic evidence in this paper, while the LIBERO material evaluates the embodied interface under completed VLA rollouts. Appendix D.2 gives the corresponding extraction, metric, carrier, and reporting details.

4.2 Protocol and Metrics

We compare three primary planning methods across four open-source LLM backbones: Qwen3-8B, Qwen2.5-7B-Instruct, LLaMA-3-8B-Instruct, and Mistral-7B-Instruct v0.3. The Direct BT baseline performs one-step direct BT generation. The

ActionSeq baseline generates an action sequence first and then compiles it into a BT. HiBT is the full hierarchical concept planning pipeline.

To isolate the contributions of symbolic repair and stable-subgoal construction, we also evaluate two internal variants. The no-repair variant disables symbolic repair, while the no-stable-subgoal variant keeps repair enabled but removes stable-subgoal construction.

For the targeted robustness study, we further compare three perturbations: the rule-only variant, which uses rule-based realization without the learned concept interface; the shuffled-concept variant, which shuffles the structural ordering of concept inputs; and the random-concept variant, which replaces the concept interface with random signals.

For all HiBT-family results on the main benchmark, we use prototype banks rebuilt under family-level exclusion from the benchmark. The same exclusion rule is also applied to Builder concept extraction and Predictor training examples. Concretely, the data pipeline first normalizes each candidate to a base problem ID and then excludes any item whose normalized family overlaps with the benchmark, including derived variants such as subtasks, subsets, and already-satisfied forms. All HiBT runs use the four-level latent concept geometry defined in Section 3, with $\mathbf{n} = (1, 4, 5, 5)$ and $N = 15$.

We report four metrics. Exec measures whether a generated BT can be parsed, compiled, and stepped by the shared symbolic runtime without structural failure. LC additionally requires consistency between the reported action sequence and the action leaves extracted from the compiled BT, and rejects executions terminating at a precondition or target-failure node. SR measures whether symbolic execution reaches the requested target state. Generation time measures planning latency, excluding model loading and simulation-time evaluation.

For LIBERO-style embodied rollouts, we keep the symbolic planning checks separate from policy execution. The planning-only setting reports Exec, LC, symbolic SR, stable-subgoal count, and planning latency over extracted benchmark requests. The VLA-guided setting reports rollout SR, subgoal completion, simulator steps, policy calls, and per-call latency. We do not compare symbolic SR directly against physical rollout SR because the latter also depends on the low-level carrier, perception preprocessing, and simulator horizon.

4.3 Main Results

Table 2 compares Direct BT, ActionSeq, and HiBT on the 60-instance main benchmark.

Backbone	Method	Exec	LC	SR	Time (s)
Qwen3-8B	Direct BT	60/60	31/60	29/60	22.54
Qwen3-8B	ActionSeq	60/60	37/60	37/60	3.17
Qwen3-8B	HiBT	60/60	60/60	60/60	0.45
Qwen2.5-7B	Direct BT	60/60	22/60	18/60	14.22
Qwen2.5-7B	ActionSeq	60/60	43/60	45/60	2.62
Qwen2.5-7B	HiBT	60/60	60/60	60/60	0.35
LLaMA-3-8B	Direct BT	58/60	26/60	24/60	26.84
LLaMA-3-8B	ActionSeq	60/60	25/60	25/60	3.11
LLaMA-3-8B	HiBT	60/60	60/60	60/60	0.37
Mistral-7B	Direct BT	40/60	23/60	19/60	28.53
Mistral-7B	ActionSeq	34/60	17/60	18/60	3.60
Mistral-7B	HiBT	60/60	60/60	60/60	0.39

Table 2: Main multi-backbone results on the 60-instance assembly benchmark.

HiBT achieves 60/60 Exec, LC, and SR for all four evaluated backbones, while Direct BT and ActionSeq exhibit substantially lower LC and SR, particularly for LLaMA-3-8B-Instruct and Mistral-7B-Instruct v0.3. ActionSeq improves over Direct BT in several settings, indicating that an action-level intermediate representation is useful, but remains below HiBT in logical coherence and task success. HiBT also has the lowest average generation latency for every backbone, reducing planning time to 0.35–0.45 seconds. The corresponding domain-level breakdown is provided in Appendix E.

4.4 Difficulty Results

To examine how performance differences vary with task difficulty, we aggregate results across the four backbones within each difficulty bucket. Table 3 reports Exec, LC, SR, and generation time aggregated across the four backbones for each difficulty bucket; each denominator equals the number of instances in the bucket multiplied by four.

HiBT attains 100% LC and SR in every benchmark difficulty bucket, including the instances labeled very hard. The largest baseline gaps occur in the hard and very hard buckets. This pattern indicates that the performance difference is most pronounced in the more dependency-intensive subsets of the evaluated benchmark.

4.5 Realization Ablations

We next evaluate whether the concept interface alone is sufficient, or whether symbolic realization

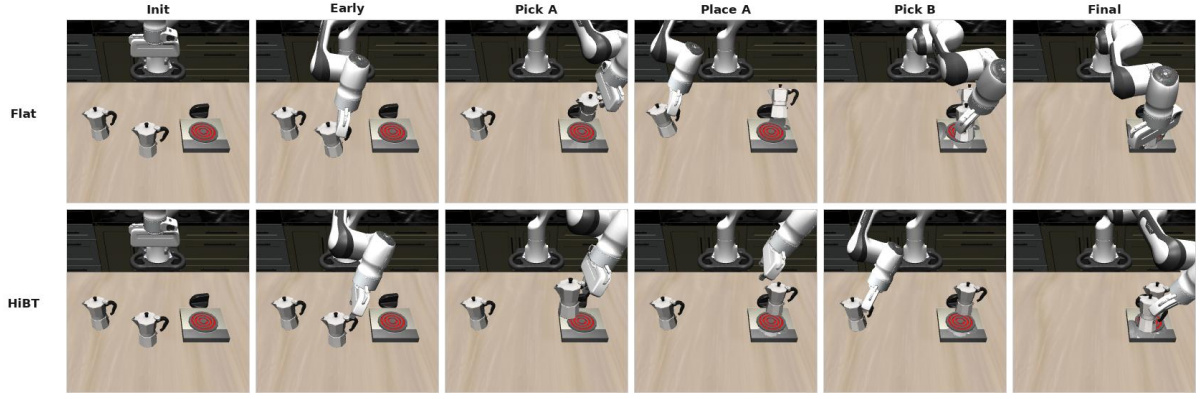


Figure 3: Task-8 LIBERO-Long process comparison. The flat-instruction rollout and HiBT-guided rollout are shown with aligned phase samples, illustrating how hierarchical subgoal guidance changes the execution trajectory on the same long-horizon manipulation task.

Bucket	Method	Exec	LC	SR	Time (s)
Easy	Direct BT	80/88	36/88	36/88	18.63
Easy	ActionSeq	76/88	52/88	54/88	2.50
Easy	HiBT	88/88	88/88	88/88	0.41
Medium	Direct BT	48/52	35/52	30/52	22.43
Medium	ActionSeq	44/52	25/52	26/52	3.40
Medium	HiBT	52/52	52/52	52/52	0.35
Hard	Direct BT	40/44	19/44	14/44	27.37
Hard	ActionSeq	40/44	21/44	21/44	3.72
Hard	HiBT	44/44	44/44	44/44	0.41
Very hard	Direct BT	50/56	12/56	10/56	27.18
Very hard	ActionSeq	54/56	24/56	24/56	3.34
Very hard	HiBT	56/56	56/56	56/56	0.39

Table 3: Difficulty-bucket results aggregated across all four backbones.

is necessary in addition to the learned concept interface. Table 4 compares the full pipeline against no-repair and no-stable-subgoal ablations.

Backbone	Variant	Exec	LC	SR	Time (s)
Qwen3-8B	No repair	60/60	24/60	24/60	0.40
Qwen3-8B	No stable subgoals	60/60	38/60	38/60	0.85
Qwen3-8B	HiBT	60/60	60/60	60/60	0.45
Qwen2.5-7B	No repair	60/60	38/60	38/60	0.67
Qwen2.5-7B	No stable subgoals	60/60	45/60	45/60	0.74
Qwen2.5-7B	HiBT	60/60	60/60	60/60	0.35
LLaMA-3-8B	No repair	60/60	13/60	13/60	0.72
LLaMA-3-8B	No stable subgoals	60/60	30/60	30/60	0.74
LLaMA-3-8B	HiBT	60/60	60/60	60/60	0.37
Mistral-7B	No repair	60/60	25/60	25/60	0.73
Mistral-7B	No stable subgoals	60/60	30/60	30/60	0.75
Mistral-7B	HiBT	60/60	60/60	60/60	0.39

Table 4: Ablation of symbolic realization components on the 60-instance assembly benchmark.

Both ablations preserve Exec at 60/60 but reduce LC and SR across all four backbones. On this benchmark, the latent concept interface alone therefore does not appear sufficient. Removing symbolic repair reduces LC and SR to 13/60–38/60 across the four backbones. The no-stable-subgoal

variant remains stronger than the no-repair variant but still falls below the full HiBT pipeline on all four backbones. This suggests that the LC and SR improvements of the full system depend on both symbolic repair and stable-subgoal construction.

4.6 Interface Perturbations

Finally, we test whether performance depends on the learned hierarchical interface rather than on deterministic realization rules alone. This study uses Qwen2.5-7B-Instruct on the same 60-instance benchmark.

Removing or perturbing the learned interface reduces LC and SR while preserving structural executability. If the learned concept interface is removed and only rules are kept, performance drops from 60/60 to 12/60 on both LC and SR. Replacing the concept interface with random signals lowers performance to 18/60, and the shuffled-concept perturbation reduces it to 53/60. Together, the ablations indicate that HiBT’s performance in this benchmark depends on both the learned hierarchical interface and deterministic dependency-closing realization.

4.7 Embodied Transfer

We next evaluate the same planning interface in LIBERO manipulation rollouts using a pi0.5/OpenPI low-level carrier. This setting separates symbolic planning from embodied execution: HiBT supplies dependency-aware subgoals and BT structure, while the VLA policy executes low-level actions from visual observations. Each LIBERO-Long method is evaluated on 10 tasks with 120 trials per task.

Table 5 shows that HiBT reaches 1070/1200

Method	Interface	SR $\uparrow$	Subgoal SR $\uparrow$	Steps $\downarrow$	Policy Calls $\downarrow$	Plan ms $\downarrow$
<i>Reference methods</i>						
Flat instruction	none	907/1200 (75.6)	N/A	418	98.7	N/A
Action sequence	flat action/subgoal list	943/1200 (78.6)	81.3	401	94.2	1760
Manual subgoal upper	BDDL/manual subgoals	1089/1200 (90.8)	93.4	368	97.8	N/A
<i>HiBT variants and controls</i>						
HiBT	learned concepts + BT + repair	1070/1200 (89.2)	92.1	365	88.7	198
HiBT without stable	concepts + sequence only	861/1200 (71.8)	76.4	461	92.8	186
HiBT without retry	concepts + no fallback	896/1200 (74.7)	83.1	411	89.5	184
Wrong-order control	shuffled subgoals	447/1200 (37.3)	44.2	497	91.1	181

Table 5: Complete LIBERO-Long rollout table. Bold marks the best value among Flat instruction, Action sequence, and HiBT, excluding manual and diagnostic controls.

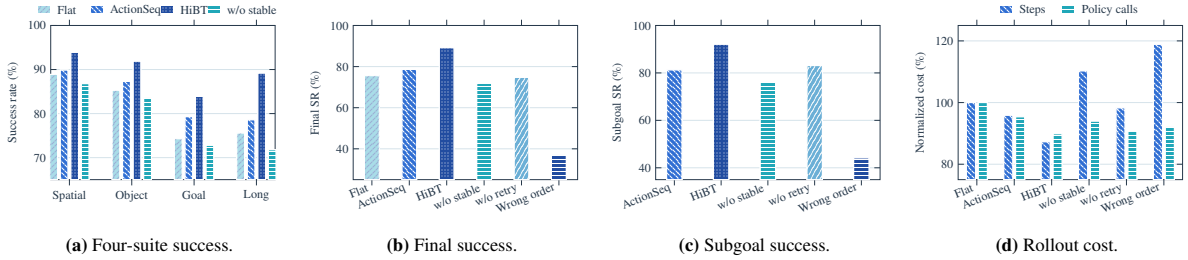


Figure 4: LIBERO rollout summary. The plots visualize the main embodied evaluation from Table 5 and the four-suite rollout summary in Appendix D.2. HiBT improves final and subgoal success while reducing normalized simulator steps and policy calls on LIBERO-Long.

Variant	Exec	LC	SR	Gen. time (s)
HiBT	60/60	60/60	60/60	0.35
Rule-only realization	60/60	12/60	12/60	0.35
Shuffled concepts	60/60	53/60	53/60	0.34
Random concepts	60/60	18/60	18/60	0.35

Table 6: Interface perturbation study on the 60-instance benchmark with Qwen2.5-7B-Instruct.

(89.2%) final-goal success, compared with 907/1200 (75.6%) for flat instruction and 943/1200 (78.6%) for action-sequence decomposition. HiBT also reduces mean rollout length from 418 to 365 simulator steps and policy calls from 98.7 to 88.7 relative to the flat baseline. Planning-only evaluation on 40 LIBERO symbolic tasks further yields 40/40 symbolic success with 40/40 logical coherence, matching the oracle pyramid upper bound (Appendix D.2). Figure 4 summarizes the same rollout trends across suite coverage, final success, subgoal completion, and normalized execution cost.

The custom LIBERO-Hier suite tests whether final success coincides with process constraints such as ordering and distractor control. As shown in Appendix Table 17, HiBT achieves 867/1200 (72.3%) strict success, compared with 363/1200 (30.3%) for flat instruction and 507/1200 (42.3%) for ActionSeq. These results suggest that the learned in-

terface and symbolic realization are useful not only for symbolic BT synthesis, but also for structuring embodied VLA execution under longer-horizon dependency constraints. Appendix Figure 5 gives the corresponding mechanism and stress-suite visual summary.

5 Conclusion

This paper studies BT synthesis for language-conditioned robotic assembly and investigates hierarchical concept planning as an alternative to direct next-token BT generation for language-conditioned robotic assembly. The proposed method separates planning into a learned hierarchical interface and a deterministic symbolic realization stage that closes local dependencies before BT compilation. Experiments on the 60-instance assembly benchmark show consistently higher logical coherence and task success, together with lower generation latency, than the evaluated direct-generation and action-sequence baselines; HiBT also achieves full executability on all tested backbones. These results suggest that executable BT synthesis in the evaluated assembly setting benefits from combining structured intermediate interfaces with deterministic realization mechanisms.

Limitations

This work has several limitations. First, the empirical evidence is grounded in a single benchmark setting centered on symbolic robotic assembly tasks. The results therefore support the proposed interface-and-realization framing only within the evaluated task distribution, and they should not be read as evidence that the same design will transfer unchanged to broader embodied planning settings, richer perception inputs, or substantially different robot domains.

Second, the current method is not a purely learned planner. Its performance depends on deterministic symbolic synthesis/repair rules that encode domain-specific assumptions about tool use, object handling, and local dependency closure. Consequently, part of the observed reliability derives from symbolic prior structure rather than from concept prediction alone. As a result, extending the approach to new domains may require nontrivial symbolic adaptation.

Third, the present evaluation does not disentangle all internal components of the symbolic realization stage at a fine-grained level. For example, the current ablations show that both symbolic repair and stable-subgoal construction matter empirically, but they do not yet establish the independent contribution of every repair sub-mechanism or isolate which parts of stable-subgoal construction matter most. The current evidence therefore supports the contribution of symbolic realization as a whole more strongly than it supports a detailed ranking of its internal design choices.

Finally, the paper studies BT synthesis under symbolic world-state inputs rather than under raw perceptual observations. This keeps the planning problem well defined and allows precise evaluation of executability and logical coherence, but it also leaves open how errors in perception, grounding, or state estimation would interact with the proposed planning interface in a more fully embodied setting.

The LIBERO rollout results should also be interpreted within their execution substrate. They use a fixed pi0.5/OpenPI low-level carrier, fixed camera preprocessing, and a simulator horizon rather than policy learning or real-world deployment. The custom LIBERO-Hier suite is useful for testing process constraints, but it is not part of the original LIBERO benchmark distribution. These results therefore support the value of the proposed planning interface for guiding an existing VLA carrier,

while leaving open how the same interface interacts with other carriers, perception stacks, and physical robot deployments.

Ethical Considerations

This work studies language-conditioned BT synthesis and robotic execution using simulation, symbolic task states, and offline demonstration data. It does not involve human-subject data, private personal information, or deployment on physical users. Nevertheless, HiBT can alter the behavior of a base execution policy by changing high-level action ordering and recovery structure. Any deployment beyond simulation should therefore include explicit checks on correction magnitude, workspace safety, tool changes, object-release events, and gripper state transitions before execution on physical hardware.

References

- Ido Amos, Avi Caciularu, Mor Geva, Amir Globerson, Jonathan Herzig, Lior Shani, and Idan Szpektor. 2026. [Latent Reasoning With Supervised Thinking States](#). *Preprint*, arXiv:2602.08332.
- Jicong Ao, Fan Wu, Yansong Wu, Abdalla Swiki, and Sami Haddadin. 2025. [LLM-as-BT-Planner: Leveraging LLMs For Behavior Tree Generation In Robot Task Planning](#). In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1233–1239. IEEE.
- Bikramjit Banerjee. 2018. [Autonomous Acquisition Of Behavior Trees For Robot Control](#). In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3460–3467. IEEE Press.
- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2023a. [Program Of Thoughts Prompting: Disentangling Computation From Reasoning For Numerical Reasoning Tasks](#). *Transactions on Machine Learning Research*.
- Xinglin Chen, Yishuai Cai, Yunxin Mao, Minglong Li, Wenjing Yang, Weixia Xu, and Ji Wang. 2024. [Integrating Intent Understanding And Optimal Behavior Planning For Behavior Tree Generation From Human Instructions](#). In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, pages 6832–6840.
- Yaran Chen, Wenbo Cui, Yuanwen Chen, Mining Tan, Xinyao Zhang, Dongbin Zhao, and He Wang. 2023b. [RoboGPT: An Intelligent Agent Of Making Embodied Long-Term Decisions For Daily Instruction Tasks](#). *Preprint*, arXiv:2311.15649.

645	Michele Colledanchise, Ramviyas Parasuraman, and	Robotic Applications. In <i>2023 IEEE International</i>	702
646	Petter Ogren. 2019. Learning Of Behavior Trees For	<i>Conference on Robotics and Automation (ICRA)</i> ,	703
647	Autonomous Agents . <i>IEEE Transactions on Games</i> ,	pages 5807–5813. IEEE.	704
648	11(2):183–189.		
649	Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey	Matteo Iovino, Edvards Scukins, Jonathan Styrud, Pet-	705
650	Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan	ter Ogren, and Christian Smith. 2020. A Survey	706
651	Wahid, Jonathan Tompson, Quan Vuong, Tianhe	Of Behavior Trees In Robotics And AI . <i>Preprint</i> ,	707
652	Yu, Wenlong Huang, Yevgen Chebotar, Pierre Ser-	arXiv:2005.05842.	708
653	manet, Daniel Duckworth, Sergey Levine, Vincent		
654	Vanhoucke, Karol Hausman, Marc Toussaint, Klaus	Riccardo Andrea Izzo, Gianluca Bardaro, and Matteo	709
655	Greff, and 3 others. 2023. PaLM-E: An Embodied	Matteucci. 2024. BTGenBot: Behavior Tree Genera-	710
656	Multimodal Language Model . In <i>Proceedings of the</i>	tion For Robotic Tasks With Lightweight LLMs . In	711
657	<i>40th International Conference on Machine Learning</i> ,	<i>2024 IEEE/RSJ International Conference on Intelli-</i>	712
658	volume 202 of <i>Proceedings of Machine Learning</i>	<i>gent Robots and Systems (IROS)</i> , pages 9684–9690.	713
659	<i>Research</i> , pages 8469–8488. PMLR.	IEEE.	714
660	Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon,	Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao	715
661	Pengfei Liu, Yiming Yang, Jamie Callan, and Gra-	Fu, Kyle Richardson, Peter Clark, and Ashish Sabhar-	716
662	ham Neubig. 2023. PAL: Program-aided Language	wal. 2023. Decomposed Prompting: A Modular Ap-	717
663	Models . In <i>Proceedings of the 40th International</i>	proach For Solving Complex Tasks . In <i>The Eleventh</i>	718
664	<i>Conference on Machine Learning</i> , pages 10764–	<i>International Conference on Learning Representa-</i>	719
665	10799. PMLR.	<i>tions</i> .	720
666	Runquan Gui, Zhihai Wang, Jie Wang, Chi Ma, Huil-	Shen Li, Daehyung Park, Yoonchang Sung, Julie A.	721
667	ing Zhen, Mingxuan Yuan, Jianye Hao, Defu Lian,	Shah, and Nicholas Roy. 2021. Reactive Task And	722
668	Enhong Chen, and Feng Wu. 2025. HyperTree Plan-	Motion Planning Under Temporal Logic Specifica-	723
669	ning: Enhancing LLM Reasoning Via Hierarchical	<i>Preprint</i> , arXiv:2103.14464.	724
670	Thinking . <i>Preprint</i> , arXiv:2505.02322.		
671	Shibo Hao, Yi Gu, Haodi Ma, Joshua Hong, Zhen	Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol	725
672	Wang, Daisy Wang, and Zhiting Hu. 2023. Reason-	Hausman, Brian Ichter, Pete Florence, and Andy	726
673	ing With Language Model Is Planning With World	Zeng. 2023. Code As Policies: Language Model	727
674	Model . In <i>Proceedings of the 2023 Conference on</i>	Programs For Embodied Control . In <i>2023 IEEE In-</i>	728
675	<i>Empirical Methods in Natural Language Processing</i> ,	<i>ternational Conference on Robotics and Automation</i>	729
676	pages 8154–8173, Singapore. Association for Com-	(<i>ICRA</i>), pages 9493–9500. IEEE.	730
677	putational Linguistics.		
678	Rishi Hazra, Pedro Zuidberg Dos Martires, and Luc	Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu,	731
679	De Raedt. 2024. SayCanPay: Heuristic Planning	Shiqi Zhang, Joydeep Biswas, and Peter Stone.	732
680	With Large Language Models Using Learnable Do-	2023. LLM+P: Empowering Large Language Mod-	733
681	main Knowledge . <i>Proceedings of the AAAI Confer-</i>	els With Optimal Planning Proficiency . <i>Preprint</i> ,	734
682	<i>ence on Artificial Intelligence</i> , 38(18):20123–20133.	arXiv:2304.11477.	735
683	Xingshuai Huang, Derek Li, Bahareh Nikpour, and	Artem Lykov and Dzmitry Tsetserukou. 2023. LLM-	736
684	Parsa Omid. 2026. Hierarchical Chain-of-Thought	BRAIn: AI-driven Fast Generation Of Robot Be-	737
685	Prompting: Enhancing LLM Reasoning Performance	haviour Tree Based On Large Language Model .	738
686	And Efficiency . <i>Preprint</i> , arXiv:2604.00130.	<i>Preprint</i> , arXiv:2305.19352.	739
687	Brian Ichter, Anthony Brohan, Yevgen Chebotar,	Qing Lyu, Shreya Havaldar, Adam Stein, Li Zhang,	740
688	Chelsea Finn, Karol Hausman, Alexander Herzog,	Delip Rao, Eric Wong, Marianna Apidianaki, and	741
689	Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, Ryan	Chris Callison-Burch. 2023. Faithful Chain-of-	742
690	Julian, Dmitry Kalashnikov, Sergey Levine, Yao Lu,	Thought Reasoning . In <i>Proceedings of the 13th In-</i>	743
691	Carolina Parada, Kanishka Rao, Pierre Sermanet,	<i>ternational Joint Conference on Natural Language</i>	744
692	Alexander T. Toshev, Vincent Vanhoucke, and 26	<i>Processing and the 3rd Conference of the Asia-Pacific</i>	745
693	others. 2023. Do As I Can, Not As I Say: Grounding	<i>Chapter of the Association for Computational Lin-</i>	746
694	Language In Robotic Affordances . In <i>Proceedings</i>	<i>guistics (Volume 1: Long Papers)</i> , pages 305–329.	747
695	<i>of The 6th Conference on Robot Learning</i> , volume	Association for Computational Linguistics.	748
696	205 of <i>Proceedings of Machine Learning Research</i> ,		
697	pages 287–318. PMLR.	Sergio Merino-Fidalgo, Celia Sánchez-Girón, Eduardo	749
698	Matteo Iovino, Julian Förster, Pietro Falco, Jen Jen	Zalama, Jaime Gómez-García-Bermejo, and Jaime	750
699	Chung, Roland Siegwart, and Christian Smith. 2023.	Duque-Domingo. 2025. Behavior Tree Genera-	751
700	On The Programming Effort Required To Gener-	tion And Adaptation For A Social Robot Control	752
701	ate Behavior Trees And Finite State Machines For	With LLMs . <i>Robotics and Autonomous Systems</i> ,	753
		194:105165.	754
		Krishan Rana, Jesse Haviland, Sourav Garg, Jad Abou-	755
		Chakra, Ian Reid, and Niko Suenderhauf. 2023. Say-	756
		Plan: Grounding Large Language Models Using 3D	757

758	Scene Graphs For Scalable Robot Task Planning . In	Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei,	813
759	<i>Proceedings of The 7th Conference on Robot Learn-</i>	Nathan Scales, Xuezhi Wang, Dale Schuurmans,	814
760	<i>ing</i> , volume 229 of <i>Proceedings of Machine Learning</i>	Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi.	815
761	<i>Research</i> , pages 23–72. PMLR.	2023. Least-to-Most Prompting Enables Complex	816
762	Xuan Shen, Yizhou Wang, Yufa Zhou, Xiangxi Shi,	Reasoning In Large Language Models . In <i>The</i>	817
763	Pu Zhao, Yanzhi Wang, and Jiuxiang Gu. 2025. Ef-	<i>Eleventh International Conference on Learning Rep-</i>	818
764	ficient Reasoning With Hidden Thinking . <i>Preprint</i> ,	<i>resentations</i> .	819
765	arXiv:2501.19201.		
766	Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit	Haotian Zhou, Yunhan Lin, Longwu Yan, Jihong Zhu,	820
767	Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox,	and Huasong Min. 2024. LLM-BT: Performing	821
768	Jesse Thomason, and Animesh Garg. 2023. Prog-	Robotic Adaptive Tasks Based On Large Language	822
769	Prompt: Program Generation For Situated Robot	Models And Behavior Trees . In <i>2024 IEEE Inter-</i>	823
770	Task Planning Using Large Language Models . <i>Au-</i>	<i>national Conference on Robotics and Automation</i>	824
771	<i>tonomous Robots</i> , 47(8):999–1012.	(ICRA), pages 16655–16661. IEEE.	825
772	Chan Hee Song, Jiaman Wu, Clayton Washington,		
773	Brian M. Sadler, Wei-Lun Chao, and Yu Su. 2023.		
774	LLM-Planner: Few-Shot Grounded Planning For Em-		
775	bodied Agents With Large Language Models . In <i>Pro-</i>		
776	<i>ceedings of the IEEE/CVF International Conference</i>		
777	<i>on Computer Vision (ICCV)</i> , pages 2998–3009.		
778	DiJia Su, Hanlin Zhu, Yingchen Xu, Jiantao Jiao,		
779	Yuandong Tian, and Qinqing Zheng. 2025. To-		
780	ken Assorted: Mixing Latent And Text Tokens For		
781	Improved Language Model Reasoning . <i>Preprint</i> ,		
782	arXiv:2502.03275.		
783	Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yun-		
784	shi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023a.		
785	Plan-and-Solve Prompting: Improving Zero-Shot		
786	Chain-of-Thought Reasoning By Large Language		
787	Models . In <i>Proceedings of the 61st Annual Meeting</i>		
788	<i>of the Association for Computational Linguistics (Vol-</i>		
789	<i>ume 1: Long Papers)</i> , pages 2609–2634. Association		
790	for Computational Linguistics.		
791	Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le,		
792	Ed Chi, Sharan Narang, Aakanksha Chowdhery, and		
793	Denny Zhou. 2023b. Self-Consistency Improves		
794	Chain Of Thought Reasoning In Language Models .		
795	In <i>The Eleventh International Conference on Learn-</i>		
796	<i>ing Representations</i> .		
797	Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten		
798	Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and		
799	Denny Zhou. 2022. Chain-of-Thought Prompting		
800	Elicits Reasoning In Large Language Models . In		
801	<i>Advances in Neural Information Processing Systems</i> ,		
802	volume 35, pages 24824–24837.		
803	Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran,		
804	Thomas L. Griffiths, Yuan Cao, and Karthik		
805	Narasimhan. 2023a. Tree Of Thoughts: Deliber-		
806	ate Problem Solving With Large Language Models .		
807	<i>Preprint</i> , arXiv:2305.10601.		
808	Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak		
809	Shafran, Karthik Narasimhan, and Yuan Cao. 2023b.		
810	ReAct: Synergizing Reasoning And Acting In Lan-		
811	guage Models . In <i>The Eleventh International Confer-</i>		
812	<i>ence on Learning Representations</i> .		

A Benchmark Details

The 60-instance benchmark used in the main paper is intentionally dependency-heavy and is not balanced across task families; most instances come from Gearset-1, while the remaining tasks cover Gearset, Chair, and Lamp. The difficulty labels are used only as analysis buckets and are not given to the planner. For all HiBT-family evaluations on the 60-instance benchmark, the prototype bank, Builder concept extraction data, and Predictor training data are filtered with family-level exclusion. The pipeline first maps each candidate to a normalized base problem ID and then excludes any item whose normalized family overlaps with the benchmark request set. This filtering also covers derived forms such as subtasks, subsets, and already-satisfied variants, so that retrieval and training do not reuse near-duplicates of the evaluation problems.

B Formal Details

This section expands the formal definitions used in Section 3 and provides the three algorithms used by HiBT. The notation follows the symbolic conventions in the main text: $\rho_{\mathcal{D}}$ denotes initial realization, $\kappa_{\mathcal{D}}$ denotes dependency closure, $\zeta_{\mathcal{D}}$ denotes single-predicate repair, Λ denotes lifting, and C denotes deterministic BT compilation.

B.1 Symbolic Assembly Domain

The symbolic assembly domain is $\mathcal{D} = (\mathcal{Q}, \mathcal{A}, \mathcal{P}, \mathcal{E}^+, \mathcal{E}^-)$, where $\mathcal{Q}$ contains grounded symbolic predicates over tool occupancy, object grasp, tool availability, and assembly relations, and $\mathcal{A}$ contains the grounded manipulation actions in the domain action inventory. For action a , the sets P_a , E_a^+ , and E_a^- denote preconditions, positive effects, and negative effects. The transition function applies only when the action preconditions are satisfied:

$$\delta(s, a) = (s \setminus E_a^-) \cup E_a^+, \quad P_a \subseteq s. \quad (17)$$

This domain definition allows structural executability and goal achievement to be evaluated separately: a schema-valid BT may still fail to reach the target state if its dependency structure is incomplete.

B.2 Plan Pyramid and Graph

The repaired plan pyramid is

$$P = (\gamma, \mathcal{S}, \mathcal{M}, \pi), \quad m_k = (q_k, a_k, U_k), \quad (18)$$

where γ is the final target predicate, $\mathcal{S}$ is the stable subgoal set, $\mathcal{M}$ is the set of grounded method nodes, and π is a linearized action sequence retained for reporting and action-leaf consistency evaluation. The dependency graph is

$$\mathcal{G}_P = (V_P, E_P), \quad V_P = \{\gamma\} \cup \mathcal{S}, \quad (19)$$

with $(r, q) \in E_P$ if and only if $r \in U_{m(q)}$. The compiler assumes this dependency graph is acyclic. Stable subgoals are conditions that can be checked or re-established under repeated BT ticking, while method nodes encode the grounded action that achieves a goal or stable subgoal and the support conditions required by that action. For a condition $q \in V_P$, let

$$m(q) = \arg \min_{m_k \in \mathcal{M}: q_k = q} k, \quad U(q) = U_{m(q)}. \quad (20)$$

If multiple candidate method nodes achieve the same predicate, this selector uses the first repaired method in the realized sequence; unresolved ties are therefore fixed before BT compilation. The induced partial order is

$$r \prec_P q \iff (r, q) \in E_P^+, \quad (21)$$

where E_P^+ is the transitive closure of E_P . Acyclicity of $\mathcal{G}_P$ is the condition that makes this order well defined and allows the recursive compiler in Algorithm 3 to terminate.

B.3 Concept Training

The concept geometry is fixed across experiments:

$$\begin{aligned} \mathbf{Z}^{(\ell)} &\in \mathbb{R}^{n_\ell \times d}, \\ \mathbf{n} &= (n_0, n_1, n_2, n_3), \\ N &= \sum_{\ell=0}^3 n_\ell. \end{aligned} \quad (22)$$

In this work, $\mathbf{n} = (1, 4, 5, 5)$ and $N = 15$. Builder supervision produces target pyramids $\mathbf{C}^* = h_\phi(u, s_0, P^*)$, and the Predictor produces $\hat{\mathbf{Z}} = f_\theta(u, s_0)$. In this instantiation, concept training uses per-level MSE:

$$\mathcal{L}_Z = \frac{1}{4} \sum_{\ell=0}^3 \frac{1}{n_\ell} \sum_{i=1}^{n_\ell} \left\| \hat{\mathbf{z}}_i^{(\ell)} - \mathbf{c}_i^{*(\ell)} \right\|_2^2. \quad (23)$$

The auxiliary token loss is teacher-forced cross entropy on the annotated solution sequence $\mathbf{y}$:

$$\mathcal{L}_\pi = - \sum_{t=1}^{|\mathbf{y}|} \log p_\theta(y_t \mid u, s_0, \mathbf{C}^*, y_{<t}). \quad (24)$$

The total loss is $\mathcal{L} = \lambda_Z \mathcal{L}_Z + \lambda_\pi \mathcal{L}_\pi$, where $\mathcal{L}_Z$ and $\mathcal{L}_\pi$ correspond to the concept reconstruction and symbolic-sequence losses above. At inference time, HiBT uses the predicted concept pyramid for interface decoding; it does not rely on generating a free-form solution trace.

B.4 Interface Decoding

The prototype bank is

$$\mathcal{B} = \{b_j = (\mathbf{C}_j, \tau_j, \Sigma_j)\}_{j=1}^M. \quad (25)$$

Each entry stores a Builder concept pyramid, a structural template, and the associated annotated plan schema. Family-level exclusion is applied before prototype-bank construction and concept-model data preparation, preventing evaluation-family near-duplicates from appearing in either retrieval entries or concept supervision data. Template matching computes

$$s_\tau(b_j) = \frac{1}{4} \sum_{\ell=0}^3 \cos \left(\mathbf{v}(\hat{\mathbf{Z}}^{(\ell)}), \mathbf{v}(\mathbf{C}_j^{(\ell)}) \right), \quad (26)$$

selects $j^* = \arg \max_j s_\tau(b_j)$, and sets $\tau^* = \tau_{j^*}$. Slot-level retrieval then selects $r_i^{(\ell)}$ from the slot bank associated with τ^* , yielding $\Xi = (\tau^*, \mathcal{R})$.

B.5 Dependency Closure

Dependency closure is defined over action preconditions. For an action a and state s ,

$$\Delta(a, s) = P_a \setminus s. \quad (27)$$

If this set is non-empty, the closure operator applies domain-supported repair rules that establish missing predicates before the action is appended to the repaired sequence. The implementation uses deterministic rule dispatch rather than open-ended search: missing hand, tool, and part-support predicates are mapped to the supported release, detach, attach, grasp, and assembly-support steps of the symbolic assembly domain. The map ν selects the next missing predicate according to this fixed repair-rule priority. The closure process terminates when the sequence can be simulated without missing preconditions or when no supported repair rule

exists for a missing predicate; the latter case is counted as a logical-coherence or task-success failure during evaluation rather than hidden inside the compiler. More explicitly, for a candidate action sequence $\pi_0 = \langle a_k \rangle_{k=1}^K$, closure maintains a state sequence $s^{(t)}$ and a repaired sequence prefix $\pi^{*(t)}$. When processing action a_k , it repeatedly applies

$$\begin{aligned} q_t &= \nu(P_{a_k} \setminus s^{(t)}), \\ \bar{\pi}_t &= \zeta_{\mathcal{D}}(q_t, s^{(t)}), \\ s^{(t+1)} &= \delta^*(s^{(t)}, \bar{\pi}_t), \end{aligned} \quad (28)$$

until $P_{a_k} \subseteq s^{(t)}$, where δ^* denotes repeated application of the one-step transition in Eq. 17. The original action is then appended and simulated:

$$\pi^{*(t+1)} = \pi^{*(t)} \circ \langle a_k \rangle, \quad s^{(t+1)} = \delta(s^{(t)}, a_k). \quad (29)$$

Algorithm 2 gives the corresponding procedure.

B.6 BT Compilation

The BT grammar used by the compiler is

$$\begin{aligned} \mathcal{T} ::= & \sigma_{\vee}([T_i]_{i=1}^n) \mid \sigma_{\wedge}([T_i]_{i=1}^n) \\ & \mid \chi(q) \mid \psi(a) \mid \alpha(a). \end{aligned} \quad (30)$$

Compilation recursively expands each target condition into a selector subtree whose achievement branch first establishes required stable subgoals and then executes the grounded action. It compiles the dependency graph rather than the linear action sequence, which is why repaired stable subgoals become explicit control-flow structure. The leaf semantics used by the symbolic evaluator are

$$\begin{aligned} \text{eval}_s(\chi(q)) &= \text{success} \Leftrightarrow q \in s, \\ \text{eval}_s(\psi(a)) &= \text{success} \Leftrightarrow P_a \subseteq s, \\ \text{eval}_s(\alpha(a)) &= (\text{success}, \delta(s, a)) \quad \text{if } P_a \subseteq s. \end{aligned} \quad (31)$$

Selectors $\sigma_{\vee}$ return the first successful child, and sequences $\sigma_{\wedge}$ require all children to succeed in order. Thus the compiled subtree $C(q) = \sigma_{\vee}[\chi(q), B_q]$ first reuses an already-satisfied condition and otherwise executes the achievement branch defined by the repaired dependency graph.

B.7 Formal Properties

Proposition 1. If each method node contains a valid grounded action schema and $\mathcal{G}_P$ is acyclic, then compilation terminates and returns a BT that contains only the allowed node types in the grammar above. The proof follows by structural induction over the acyclic dependency graph: leaves

Algorithm 1 Hierarchical Interface Induction

Input: task instruction u , initial symbolic state s_0 , prototype bank $\mathcal{B}$

Output: hierarchical planning interface $\Xi = (\tau^*, \mathcal{R})$

```

1: Predict hierarchy,  $\hat{\mathbf{Z}} \leftarrow f_\theta(u, s_0)$ 
2: for each  $b_j = (\mathbf{C}_j, \tau_j, \Sigma_j) \in \mathcal{B}$  do
3:    $s_j \leftarrow \frac{1}{4} \sum_{\ell=0}^3 \cos(\mathbf{v}(\hat{\mathbf{Z}}^{(\ell)}), \mathbf{v}(\mathbf{C}_j^{(\ell)}))$ 
4: end for
5: Select template,  $j^* \leftarrow \arg \max_j s_j$ 
6:  $\tau^* \leftarrow \tau_{j^*}$ 
7:  $\mathcal{R} \leftarrow \emptyset$ 
8: for  $\ell \in \{0, 1, 2, 3\}$  and  $i \in [n_\ell]$  do
9:    $r_i^{(\ell)} \leftarrow \arg \max_{p \in \mathcal{B}_i^{(\ell)}(\tau^*)} \cos(\hat{\mathbf{z}}_i^{(\ell)}, \mathbf{c}(p))$ 
10:  Retrieve slot item,  $\mathcal{R} \leftarrow \mathcal{R} \cup \{r_i^{(\ell)}\}$ 
11: end for
12: return  $\Xi = (\tau^*, \mathcal{R})$ 

```

have no required stable subgoals and compile to one selector with a finite sequence branch; each internal condition compiles after its required subgoals, which are lower in the acyclic dependency order.

Proposition 2. If dependency closure ensures that every required stable subgoal of a method node can be checked or established before the corresponding action is executed, then the compiled BT represents those dependencies through explicit recursive subtrees and precondition checks. This follows because $C(q)$ recursively compiles all $r \in U_{m(q)}$ before inserting $\psi(a_{m(q)})$ and $\alpha(a_{m(q)})$. These properties concern schema validity and dependency preservation; they do not replace the empirical SR evaluation of whether symbolic execution reaches γ .

B.8 Algorithms

The pseudocode below gives the procedural form of the three method stages referenced in Section 3: interface induction (Algorithm 1), dependency-closed symbolic realization (Algorithm 2), and deterministic BT compilation (Algorithm 3).

C Ablation Interpretation

The main multi-backbone results and the concept-robustness study together suggest that the learned concept layer is carrying task-relevant planning structure rather than serving only as an auxiliary representation. If the final performance were mainly explained by deterministic symbolic rules, then the rule-only variant would be expected to remain close to the full concept pipeline. Instead, the rule-only variant drops sharply in both logical coherence and end-to-end task success, even

Algorithm 2 Symbolic Realization with Dependency Closure

Input: target predicate γ , initial state s_0 , interface Ξ , domain $\mathcal{D}$

Output: repaired plan pyramid $\tilde{P}$

```

1: Instantiate candidate,  $\pi_0 \leftarrow \rho_{\mathcal{D}}(\gamma, s_0, \Xi)$ 
2:  $\pi^* \leftarrow \langle \rangle$ 
3:  $s \leftarrow s_0$ 
4: for each  $a \in \pi_0$  do
5:    $\Delta \leftarrow P_a \setminus s$ 
6:   while  $\Delta \neq \emptyset$  do
7:      $q \leftarrow \nu(\Delta)$ 
8:     Add repair segment,  $\bar{\pi} \leftarrow \zeta_{\mathcal{D}}(q, s)$ 
9:     for each  $\bar{a} \in \bar{\pi}$  do
10:      append  $\bar{a}$  to  $\pi^*$ 
11:       $s \leftarrow \delta(s, \bar{a})$ 
12:     end for
13:    $\Delta \leftarrow P_a \setminus s$ 
14:   end while
15:   Commit original action, append  $a$  to  $\pi^*$ 
16:    $s \leftarrow \delta(s, a)$ 
17: end for
18: return  $\tilde{P} \leftarrow \Lambda(\gamma, \pi^*)$ 

```

Algorithm 3 Deterministic Plan-to-BT Compilation

Input: repaired plan pyramid $\tilde{P} = (\gamma, \mathcal{S}, \mathcal{M}, \pi)$

Output: executable Behavior Tree $\mathcal{T}$

```

1: function  $C(q)$ 
2:    $m(q) \leftarrow$  method node in  $\mathcal{M}$  that achieves  $q$ 
3:    $\mathcal{B}_q \leftarrow \langle \rangle$ 
4:   for each  $r \in U_{m(q)}$  do
5:     append  $C(r)$  to  $\mathcal{B}_q$ 
6:   end for
7:   append  $\psi(a_{m(q)})$  to  $\mathcal{B}_q$ 
8:   append  $\alpha(a_{m(q)})$  to  $\mathcal{B}_q$ 
9:   return  $\sigma_{\vee}[\chi(q), \sigma_{\wedge}[\mathcal{B}_q]]$ 
10: end function
11: return  $\mathcal{T} \leftarrow C(\gamma)$ 

```

though executability remains intact. This means that symbolic rules alone can maintain structural well-formedness, but they do not provide enough information to recover the task-specific dependencies needed for correct execution.

The perturbation results reinforce the same point. Replacing the concept interface with random signals substantially degrades performance, and even shuffling concept structure causes a visible drop relative to the full model. These patterns indicate that performance depends on preserving the learned interface structure. The learned interface constrains symbolic realization toward task-relevant subgoal structure rather than leaving all structure to downstream rules. This interpretation also clarifies the role of the concept interface in the full system: the concepts do not directly decode into a finished executable plan, but they provide a structured prior that is strong enough to shape which symbolic plan

should be realized.

The comparison between ActionSeq and HiBT addresses a different question: is any explicit intermediate representation sufficient to explain the observed performance difference between language and BT compilation? The results suggest that the answer is no. ActionSeq often improves over direct one-step BT generation, which indicates that intermediate structure is indeed helpful. However, it still remains materially below HiBT, especially in logical coherence and task success. ActionSeq exposes action order and provides a clearer path to compilation than direct BT text generation, but it does not explicitly encode which stable subgoals support which grounded achievement steps, nor does it preserve reusable dependency structure before compilation. The difficulty breakdown strengthens this interpretation: the gap between ActionSeq and HiBT becomes most salient on hard and very hard instances, where tasks contain denser dependencies and more opportunities for tool switching, subgoal interaction, and local inconsistency.

The symbolic-realization ablations show that the concept interface alone is not sufficient to determine a coherent executable plan. The no-repair variant preserves executability but loses a large amount of logical coherence and task success, indicating that interface-level structure alone does not guarantee that local dependencies, preconditions, and tool-state transitions are realized in a way that supports correct execution. The comparison between the full method and the no-stable-subgoal variant adds a more specific interpretation: removing stable-subgoal construction causes substantial degradation on all four backbones, even though the resulting plans remain executable. The current evidence therefore supports a refined conclusion: symbolic realization contributes as a whole, and stable-subgoal construction is a substantive mechanism rather than a negligible design choice.

C.1 Template and Interface Ablations

To further separate template priors from the full HiBT interface, we include a targeted ablation study on the same 60-instance benchmark using the Qwen2.5-7B backbone. This study compares four reduced settings against the full concept pipeline. retrieval-only nearest-template skips hierarchical concept construction and uses only the nearest structural template from the prototype bank before symbolic realization.

classifier-to-template predicts a template ID directly from the task goal and symbolic world state, without passing through a concept pyramid. concept-without-slot-retrieval keeps concept prediction and template scoring, but realization receives only the selected top-level template prior and does not use slot-level prototype grounding. concept-without-bank keeps the concept predictor but removes the prototype/template bank entirely, so the realization stage no longer receives bank-derived template or slot guidance and instead falls back to concept-conditioned symbolic synthesis without prototype support. The released targeted summary for this study records task success (SR) as its primary outcome.

Method	SR ↑
Qwen2.5-7B HiBT / Concept	60/60
Qwen2.5-7B concept-without-slot-retrieval	47/60
Qwen2.5-7B retrieval-only nearest-template	39/60
Qwen2.5-7B classifier-to-template	37/60
Qwen2.5-7B concept-without-bank	30/60

Table 7: Template/interface ablations on the 60-instance benchmark with Qwen2.5-7B. The released targeted summary records SR; the full HiBT result is included as a reference point.

These results help separate different sources of guidance inside the concept pipeline. First, nearest-template retrieval alone reaches only 39/60, which is well below the full 60/60 concept result; this indicates that the gain is not adequately explained by a coarse template prior alone. Second, direct classification to a template ID reaches 37/60, which is similar to retrieval-only nearest-template and therefore also falls far short of the full interface. Third, removing slot-level retrieval drops success to 47/60, showing that top-level template routing alone does not exhaust the value of the HiBT interface: slot-aware prototype grounding contributes additional planning signal. Finally, removing the bank entirely lowers success to 30/60, which shows that the bank is not merely a convenience for template lookup, but an important part of how the learned interface is converted into useful symbolic guidance. Taken together, this ablation suggests a more specific ordering: coarse template routing is not enough, slot-level retrieval adds clear value, and the full HiBT advantage comes from bank-guided, slot-aware interface construction rather than from a template prior alone.

D Protocol Details

Exec, LC, and SR are computed by the same symbolic runtime. Exec checks that the generated tree can be parsed, compiled, and evaluated without structural failure. LC is stricter: it requires executability, action-sequence consistency between the reported action sequence and the action leaves extracted from the compiled BT, and no terminal precondition or target failure node. SR measures whether the simulator reaches the requested target state.

Generation time measures planning latency rather than full process startup. For HiBT, it includes concept prediction, template scoring, slot retrieval, symbolic synthesis/repair, and BT compilation. It excludes model loading and simulation-time evaluation. Separate single-request runtime measurements from the evaluation package include model-loading and wall-clock overhead: direct BT generation takes 46.76 seconds to load and 24.77 seconds to generate for a 77.11-second wall-clock run with 17,954 MiB peak GPU memory, while the action-sequence route takes 41.92 seconds to load and 4.15 seconds to generate for a 50.83-second wall-clock run with 17,018 MiB peak GPU memory. These single-request measurements include model-loading and wall-clock overhead, so they are reported separately from the benchmark generation-latency results.

D.1 Compute Environment

Table 8 reports the compute environment used for the reported experiments. Generation and evaluation jobs are run with single-GPU execution unless otherwise noted; the multi-GPU server is used to schedule independent runs and repeated evaluations.

Component	Configuration
Compute server	8× NVIDIA RTX 5880 Ada GPUs, 48GB VRAM each
Execution mode	Single-GPU execution unless otherwise noted

Table 8: Compute setup used for the reported experiments.

D.2 LIBERO Extension

The LIBERO extension evaluates whether the same planning interface can be connected to a standard embodied manipulation benchmark. Unlike the assembly benchmark in the main paper, LIBERO has two distinct evaluation layers. The first layer extracts symbolic planning requests from benchmark

task specifications and evaluates whether the generated high-level plan is executable and logically coherent under the extracted symbolic goal structure. The second layer uses the generated subgoals to guide a VLA carrier in MuJoCo rollouts. These layers answer different questions: the first tests symbolic task decomposition, while the second tests whether the decomposition provides useful guidance to an existing low-level embodied policy. The adapter reads each LIBERO task language description together with the objects, fixtures, initial predicates, and goal predicates provided by the benchmark specification. It then normalizes goal predicates into manipulation primitives covering object placement, spatial relations, articulation state, and appliance state. The resulting request has the same role as an assembly instance in the main paper: it supplies a language-conditioned task, an initial symbolic state, and a target structure that can be passed through the HiBT interface. For planning-only evaluation, the generated BT or subgoal sequence is checked against this extracted symbolic structure. For rollout evaluation, the same high-level structure is converted into natural subgoal prompts for the VLA carrier.

The benchmark scope contains the four ten-task LIBERO suites: *LIBERO Spatial*, *LIBERO Object*, *LIBERO Goal*, and *LIBERO-10*, together with a custom stress-test suite for longer-horizon dependency cases. The custom stress suite is reported separately from standard LIBERO results because it is not part of the original benchmark distribution.

Table 9 summarizes the metrics used by the LIBERO extension. The symbolic metrics mirror the main-paper assembly metrics, while the rollout metrics account for low-level policy execution. This separation is important because a plan can satisfy the extracted symbolic goal structure but still fail during embodied execution due to policy, perception, contact, or horizon failures.

Table 10 summarizes the completed LIBERO evaluation matrix. VLA experiments use pi0.5/OpenPI with rotate180 preprocessing, a 650-step horizon, a 180-step stage budget, and a five-step replanning period.

Layer	Metric	Meaning
<i>Planning-only</i>		
Symbolic	Exec	Generated structure can be parsed and checked.
Symbolic	LC	Action leaves and reported plan remain logically coherent.
Symbolic	Symbolic SR	Extracted symbolic goal predicates are satisfied.
Symbolic	Stable SG	Number of explicit stable subgoals in the plan.
Symbolic	Plan ms	Planning latency before embodied rollout.
<i>VLA rollout</i>		
Embodied	SR	Simulator episode reaches the benchmark success condition.
Embodied	Subgoal SR	Planned subgoals are completed during rollout.
Embodied	Steps	Simulator steps used by the episode.
Embodied	Policy calls	Low-level carrier invocations.
Embodied	Latency	Mean or percentile policy-call latency.

Table 9: LIBERO extension metrics. Symbolic planning success and embodied rollout success are reported separately.

Method	Exec $\uparrow$	LC $\uparrow$	Sym. SR $\uparrow$	Stable SG	Plan ms $\downarrow$
Direct BT	40/40	31/40	31/40	N/A	16
ActionSeq	40/40	39/40	39/40	1.3	11
Concept raw	40/40	39/40	39/40	1.3	10
Concept without stable	38/40	38/40	35/40	1.1	12
Concept / HiBT	40/40	40/40	40/40	1.3	12
Oracle pyramid	40/40	40/40	40/40	1.3	10

Table 11: LIBERO planning-only evaluation on 40 symbolic tasks. No VLA rollout is performed. Symbolic SR is predicate-level success under the symbolic checker.

The embodied extension uses an existing VLA policy as the low-level carrier and tests whether HiBT-style task decomposition changes the high-level execution interface. The primary carrier in this study is a pi0.5/OpenPI LIBERO policy. OpenVLA-OFT is treated as an interface diagnostic unless its required software stack is available and full-horizon episodes complete reliably. SmolVLA is not part of the reported comparison. Camera preprocessing is fixed before policy calls so that all compared guidance variants use the same observation convention.

The rollout variants are organized around the amount and structure of guidance given to the VLA carrier. A flat VLA baseline receives only the original task instruction. An action-sequence variant converts extracted symbolic actions into sequential subgoal prompts. A concept-guided variant uses the dependency-aware subgoal structure produced by the HiBT interface. Repaired concept guidance additionally applies execution-aware ordering so that placement subgoals precede final state-changing actions when a task requires both spatial and state predicates. This design isolates whether high-level subgoal organization helps the same low-level carrier recover from long-horizon or ordering-sensitive tasks.

The LIBERO protocol is organized into request extraction, symbolic planning checks, VLA-guided

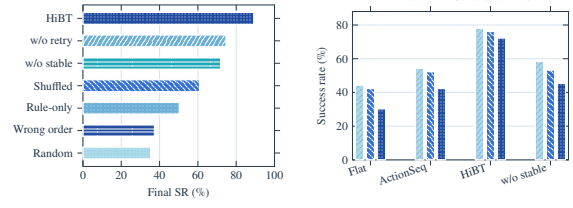


Figure 5: Mechanism and stress-suite summaries for the LIBERO extension. The left plot compares learned-interface, stable-subgoal, retry, and ordering controls on LIBERO-Long, while the right plot reports final, constraint, and strict success on the custom LIBERO-Hier suite.

rollout evaluation, carrier validation, and result aggregation. Paper-facing results should be generated from fixed evaluation manifests and completed run summaries, with visual evidence retained as supporting material rather than as a substitute for quantitative reporting. This keeps the embodied extension reproducible while avoiding dependence on implementation-specific file names or machine-specific directory layouts.

D.3 LIBERO Qualitative Visuals

E Domain Breakdown

Table 18 aggregates results by task domain across the four tested backbones. The table is a diagnostic breakdown rather than a leave-one-domain-out generalization result, because the benchmark distribution is imbalanced. HiBT attains full benchmark scores within each reported domain subset, although this diagnostic breakdown should not be interpreted as cross-domain generalization.

Domain	Method	N pooled	Exec	LC	SR
Gearset	Direct BT	4	4/4	4/4	4/4
Gearset	ActionSeq	4	4/4	4/4	4/4
Gearset	HiBT	4	4/4	4/4	4/4
Gearset-1	Direct BT	176	162/176	79/176	73/176
Gearset-1	ActionSeq	176	157/176	98/176	101/176
Gearset-1	HiBT	176	176/176	176/176	176/176
Chair	Direct BT	36	30/36	11/36	10/36
Chair	ActionSeq	36	30/36	14/36	14/36
Chair	HiBT	36	36/36	36/36	36/36
Lamp	Direct BT	24	22/24	8/24	3/24
Lamp	ActionSeq	24	23/24	6/24	6/24
Lamp	HiBT	24	24/24	24/24	24/24

Table 18: Cross-domain breakdown aggregated across the four tested backbones.

Phase	Benchmark	Methods	Budget	Reported in
<i>Symbolic checks</i>				
Planning-only	40 symbolic tasks	direct, ActionSeq, concept, oracle	40 tasks	Table 11
<i>Embodied rollouts</i>				
VLA Long	<i>LIBERO-10</i>	flat, ActionSeq, HiBT, ablations	$10 \times 120 = 1200$ ep	Table 5
Four-suite VLA	spatial / object / goal / long	flat, ActionSeq, HiBT	4×1200 ep	Table 12
Stress suite	<i>LIBERO-Hier10</i>	flat, ActionSeq, HiBT, manual	$10 \times 120 = 1200$ ep	Tables 17, 14
Latency	VLA rollouts	rollout methods	per episode	Table 16
Qualitative	<i>LIBERO-10</i> task 8	flat, ActionSeq, HiBT	30 inits	Fig. 3

Table 10: LIBERO experiment matrix. Budget is reported as tasks times trials, yielding episodes per method. The qualitative task-8 entry corresponds to the main-text visual comparison in Fig. 3.

Method	Spatial SR $\uparrow$	Object SR $\uparrow$	Goal SR $\uparrow$	Long SR $\uparrow$	Mean Steps $\downarrow$
Flat instruction	1067/1200 (88.9)	1023/1200 (85.3)	891/1200 (74.3)	907/1200 (75.6)	428
Action sequence	1079/1200 (89.9)	1047/1200 (87.3)	951/1200 (79.3)	943/1200 (78.6)	415
Manual subgoal upper	1131/1200 (94.3)	1109/1200 (92.4)	1023/1200 (85.3)	1089/1200 (90.8)	368
HiBT	1127/1200 (93.9)	1103/1200 (91.9)	1007/1200 (83.9)	1070/1200 (89.2)	372
HiBT without stable	1041/1200 (86.8)	1003/1200 (83.6)	873/1200 (72.8)	861/1200 (71.8)	446

Table 12: Four-suite LIBERO VLA rollout summary. Each suite cell contains 1200 episodes. This table is also the source for the planned four-suite result figure.

Variant	Learned	Stable	Retry	SR $\uparrow$	Subgoal SR $\uparrow$	Dominant Failure
HiBT	yes	yes	yes	1070/1200 (89.2)	92.1	residual policy / timeout
No stable checks	yes	no	yes	861/1200 (71.8)	76.4	state / order violation
No retry/fallback	yes	yes	no	896/1200 (74.7)	83.1	timeout / placement
Rule-only realization	no	yes	yes	603/1200 (50.3)	63.4	wrong decomposition
Random concept	random	yes	yes	423/1200 (35.3)	47.1	wrong decomposition
Shuffled concept	perturbed	yes	yes	729/1200 (60.8)	70.2	misplaced subgoal
Wrong-order control	yes	yes	yes	447/1200 (37.3)	44.2	precondition / order

Table 13: Mechanism ablation on LIBERO-Long. Learned, Stable, and Retry indicate enabled components. Dominant Failure is the modal first-failure stage on failed episodes.

Method	Final SR $\uparrow$	Constraint SR $\uparrow$	Strict SR $\uparrow$	Distractor Move $\downarrow$	Order Valid $\uparrow$
Flat instruction	531/1200 (44.3)	507/1200 (42.3)	363/1200 (30.3)	7.6 cm	701/1200 (58.4)
Action sequence	651/1200 (54.3)	627/1200 (52.3)	507/1200 (42.3)	5.4 cm	819/1200 (68.3)
Manual subgoal upper	983/1200 (81.9)	959/1200 (79.9)	911/1200 (75.9)	1.4 cm	1129/1200 (94.1)
HiBT	937/1200 (78.1)	913/1200 (76.1)	867/1200 (72.3)	2.0 cm	1049/1200 (87.4)
HiBT without stable	699/1200 (58.3)	639/1200 (53.3)	543/1200 (45.3)	4.9 cm	831/1200 (69.3)

Table 14: Complete custom LIBERO-Hier stress-suite table. Final SR measures only the BDDL final goal, while Strict SR requires both final-goal and process constraint success.

Benchmark	Method	Exec / Valid $\uparrow$	LC / Logic $\uparrow$	SR $\uparrow$	Cost $\downarrow$
<i>Symbolic assembly</i>					
Assembly test60	Direct BT	60/60	31/60	29/60	22.54 s
Assembly test60	ActionSeq	60/60	37/60	37/60	3.17 s
Assembly test60	HiBT / Concept	60/60	60/60	60/60	0.45 s
<i>LIBERO expansion</i>					
LIBERO planning-only	Direct BT	40/40	31/40	31/40	16 ms
LIBERO planning-only	ActionSeq	40/40	39/40	39/40	11 ms
LIBERO planning-only	HiBT / Concept	40/40	40/40	40/40	12 ms
LIBERO-Long VLA	Flat VLA	N/A	N/A	907/1200 (75.6)	18.2 s/ep
LIBERO-Long VLA	HiBT + VLA	N/A	92.1 subgoal SR	1070/1200 (89.2)	15.7 s/ep

Table 15: Cross-benchmark snapshot. Assembly uses symbolic test60, LIBERO planning-only uses 40 symbolic tasks, and LIBERO-Long VLA uses 1200 rollouts per method. Cost metrics are interpreted only within each benchmark group.

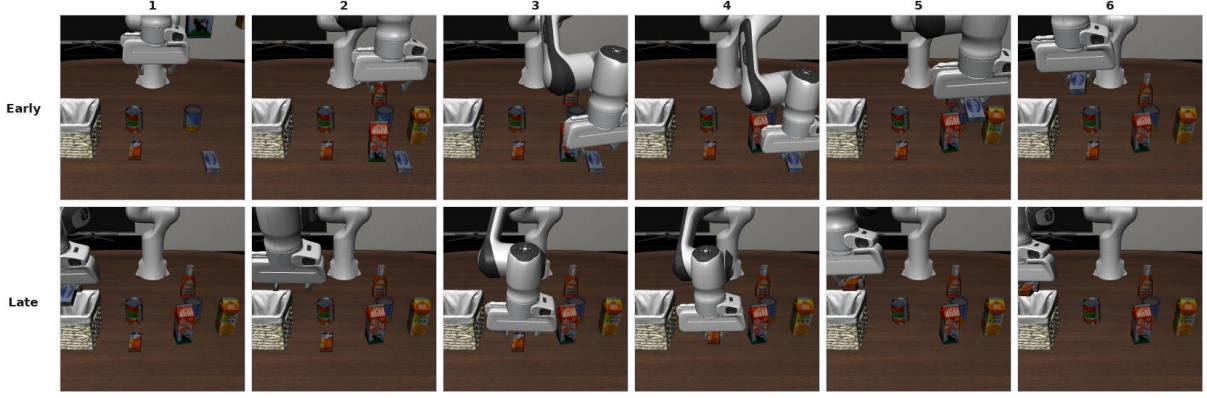


Figure 6: Appendix LIBERO-Long task-1 flat-policy process sheet. This successful non-task-8 trajectory completes the main-text task-8 comparison in Fig. 3.

Method	Planner ms ↓	P50 ms ↓	P95 ms ↓	Wall s/Ep ↓
Flat instruction	N/A	84	87	18.2
Action sequence	1760	84	87	16.3
HiBT	198	83	87	15.7
HiBT cached plan	24	83	87	15.4
Manual subgoal upper	N/A	83	87	15.5

Table 16: Latency accounting under the 120-trial rollout setting. Policy P50/P95 are per-call latency quantiles; Wall s/Ep excludes model loading.

Method	Final ↑	Constr. ↑	Strict ↑	Distr. ↓	Order ↑
Flat	44.3	42.3	30.3	7.6	58.4
ActionSeq	54.3	52.3	42.3	5.4	68.3
Manual upper	81.9	79.9	75.9	1.4	94.1
HiBT	78.1	76.1	72.3	2.0	87.4
Without stable	58.3	53.3	45.3	4.9	69.3

Table 17: Custom LIBERO-Hier stress-suite summary. Success and order-valid metrics are percentages over 1200 episodes; Distr. is mean distractor displacement in cm. Bold excludes the manual upper bound.

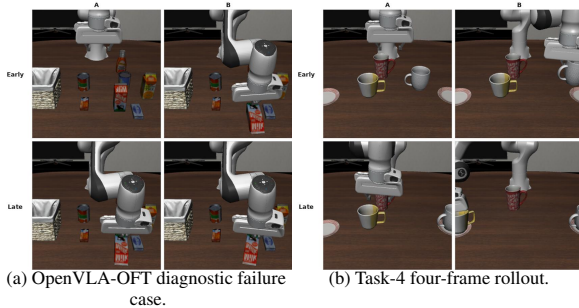


Figure 7: Appendix LIBERO rollout diagnostics. The left panel shows the OpenVLA-OFT task-1 diagnostic failure case used to assess carrier readiness; the right panel shows a compact task-4 2×2 rollout summary under the pi0.5/OpenPI carrier.

F Failure Modes

Direct BT generation and action-sequence-first planning mostly fail through hand/tool occupancy, missing equippability, malformed action arguments, or unsupported manipulation preconditions. Unrepaired concept decoding often remains structurally executable, but fails because the required stable support relations are missing. These patterns indicate that symbolic realization contributes dependency completion beyond structural BT formatting.

G Worked Example

We illustrate the full realization and compilation process using a successfully completed shaft-insertion instance. The initial state contains an occupied outward gripper and requires switching to a clamp gripper before the final insertion, so the realized manipulation first releases the already held gear, detaches the outward gripper, attaches the clamp gripper, grasps the shaft, and then inserts the shaft into the gear-base hole. The corresponding plan pyramid stores the final insertion goal at level 0 and places the required stable support conditions at level 1: the outward gripper has been cleared, the released gripper is again available, the left hand holds the clamp gripper, and the clamp gripper holds the shaft. Level 2 maps these support conditions to grounded method nodes, where the release, tool-detachment, tool-attachment, grasp, and insertion steps respectively establish the support relations needed by later steps. Level 3 preserves the readable action sequence. The compiler then turns each goal or stable subgoal into a selector with a target-check branch and a sequence branch, so repeated BT ticking can reuse already satisfied con-

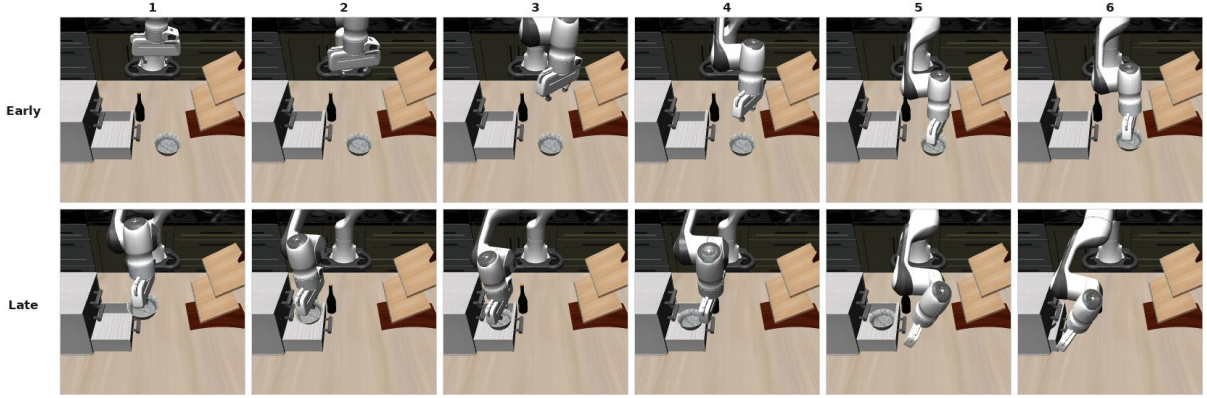


Figure 8: Appendix LIBERO-Long task-3 flat-policy process sheet. This is one of the three non-main-text 2×6 process visualizations selected from the candidate sweep tasks.

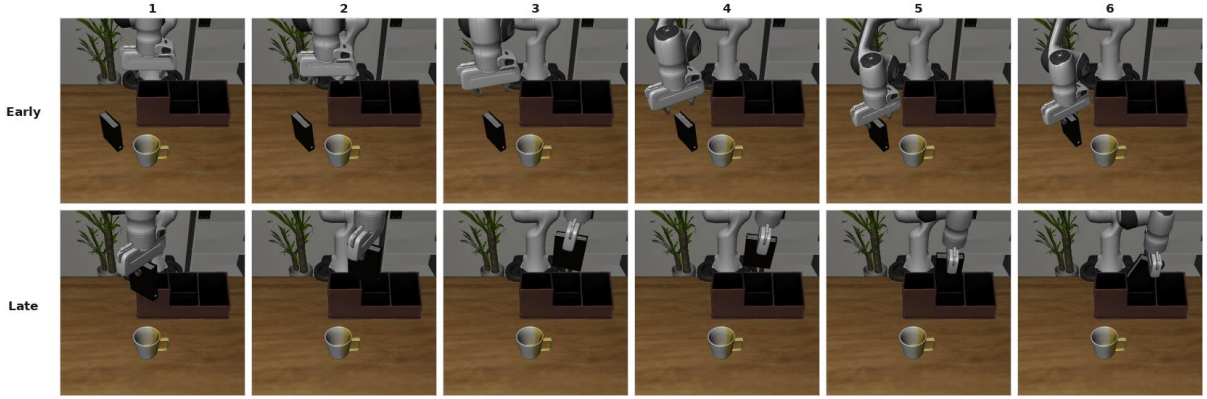


Figure 9: Appendix LIBERO-Long task-5 flat-policy process sheet. The figure adds a second rollout trajectory to the appendix so the qualitative evidence is not limited to the main-text examples.

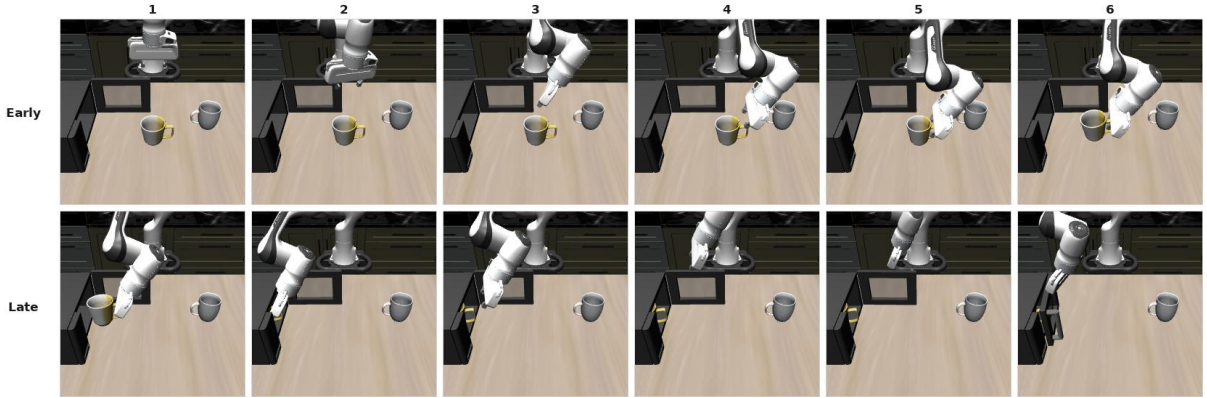


Figure 10: Appendix LIBERO-Long task-9 flat-policy process sheet. Together with the task-3 and task-5 sheets, this provides broader visual coverage across the successful sweep rollouts.

ditions while preserving the repaired dependency chain.

H Reproducibility

The supplementary evaluation package contains prompt assets, plan schemas, prototype banks, deterministic compilation code, and shared evaluation interfaces. HiBT requires a Builder checkpoint, a Predictor checkpoint, and a fixed prototype bank. Reproducing the main results consists of verifying the 60-instance request set, evaluating Direct BT, ActionSeq, and HiBT under the same symbolic simulator, and then running the decoder/compiler and concept-robustness ablations. At the appendix level, the tables are reproduced from the frozen main-result, complexity, domain, decoder/compiler, and concept-robustness summaries.